

1 = NES-VMC 算法在 NETKET 官方 API 中的复现：目的与进展

1.1 = 1. 研究目的

目标：完全基于 NetKet 框架的高层 API (MCState、VMC 驱动) 复现 **NES-VMC (Natural Excited State Variational Monte Carlo)** 算法，用于计算量子多体系统（如 H_2 分子）的前 K 个激发态能量。

要求：

- 使用 NetKet 内置的扩展希尔伯特空间 $h_i \times K$
- 使用 MCState 管理变分态与采样器
- 使用 VMC 驱动自动执行训练循环
- 最终通过训练得到的模型，对角化平均局域能量矩阵，获得基态与激发态能量

1.2 = 2. NES-VMC 算法核心思想

1.2.1 = 2.1 问题背景

在量子力学中，我们通常要求解哈密顿算符 $\hat{H}$ 的本征值问题，即找到最低的 K 个本征函数。对于量子多体系统，直接对角化哈密顿矩阵通常是不可行的，因为希尔伯特空间的维度随粒子数指数增长。

NES-VMC 将原系统前 K 个激发态的求解问题等价转化为一个“扩展系统”的基态求解问题。

1.2.2 = 2.2 扩展希尔伯特空间

设 $x = (x_1, \dots, x_N)$ 表示一组包含 N 个粒子的粒子集 (particle set)，其中 x_i 表示第 i 个粒子的状态。扩展希尔伯特空间由 K 个原系统副本张量积构成，每个配置对应 K 个组态 $\mathbf{x} = (x^1, \dots, x^K)$ 。

1.2.3 = 2.3 TOTALANSATZ 的构成

设 ψ_i 表示第 i 个 N 粒子波函数（可能未归一化），则 **TotalAnsatz** 定义为矩阵 $\Psi(\mathbf{x}) \in \mathbb{R}^{K \times K}$ 的行列式：

$$\Psi(\mathbf{x}) \equiv \det \begin{pmatrix} \psi_1(x^1) & \psi_2(x^1) & \cdots & \psi_K(x^1) \\ \psi_1(x^2) & \psi_2(x^2) & \cdots & \psi_K(x^2) \\ \vdots & \vdots & \ddots & \vdots \\ \psi_1(x^K) & \psi_2(x^K) & \cdots & \psi_K(x^K) \end{pmatrix}$$

其中：

- $\Psi(\mathbf{x}) \in \mathbb{R}^{K \times K}$ ：将所有电子集合与所有波函数结合的矩阵
- $\psi_i(x^j)$ ：第 i 个单态 Ansatz 在第 j 个粒子集上的值
- $\Psi(\mathbf{x}) = \det(\Psi(\mathbf{x}))$ ：总 Ansatz，可以看作是由 N 粒子波函数组成的未归一化 Slater 行列式

关键性质： 通过将总 Ansatz 表示为单态 Ansatz 的行列式，可以防止不同 Ansatz 坍缩到同一状态，而不需要显式要求它们正交。

1.2.4=2.4 扩展哈密顿量

定义扩展哈密顿量 $\tilde{H} = \hat{H}_1 \oplus \hat{H}_2 \oplus \cdots \oplus \hat{H}_K$ ，其中 $\hat{H}_i$ 是仅作用于第 i 个粒子集的哈密顿量。 $\tilde{H}$ 的基态能量等于原系统 $\hat{H}$ 最低 K 个能量之和，其基态波函数正是上述行列式形式的 Ψ^* 。

1.3=3. 损失函数

1.3.1=3.1 目标函数（RAYLEIGH 商）

NES-VMC 的目标函数为扩展哈密顿量关于总 Ansatz 的 Rayleigh 商：

$$\mathcal{L} = \frac{\langle \Psi | \tilde{H} | \Psi \rangle}{\langle \Psi | \Psi \rangle}$$

利用矩阵行列式引理，可以将其重写为迹形式：

$$\mathcal{L} = \frac{\langle \Psi | \tilde{H} | \Psi \rangle}{\det(S)} = \mathrm{Tr}(S^{-1} \tilde{H})$$

其中 S 为重叠矩阵：

$$S = \begin{pmatrix} \langle \psi_1 | \psi_1 \rangle & \cdots & \langle \psi_1 | \psi_K \rangle \\ \vdots & \ddots & \vdots \\ \langle \psi_K | \psi_1 \rangle & \cdots & \langle \psi_K | \psi_K \rangle \end{pmatrix}$$

1.3.2=3.2 局域能量矩阵

通过 Monte Carlo 采样，损失函数可以写成期望值形式：

$$\mathcal{L} = \mathbb{E}_{\mathbf{x} \sim \Psi^2} \left[\mathrm{Tr} \left(\Psi^{-1} (\mathbf{x}) \tilde{H} \Psi(\mathbf{x}) \right) \right]$$

定义**局域能量矩阵**为：

$$E_L(\mathbf{x}) \equiv \Psi^{-1}(\mathbf{x}) \tilde{H} \Psi(\mathbf{x})$$

这是一个 $K \times K$ 矩阵，其迹即为标量局域能量。当 $K = 1$ 时，这退化为标准 VMC 中的局域能量。

1.4=4. 梯度公式

1.4.1=4.1 标准 VMC 梯度回顾

对于基态 VMC，能量关于变分参数 θ 的梯度为：

$$\nabla_{\theta} \frac{\langle \Psi | \hat{H} | \Psi \rangle}{\langle \Psi | \Psi \rangle} = 2 \mathbb{E}_{\mathbf{x} \sim \Psi^2} \left[\left(E_L(\mathbf{x}) - \mathbb{E}_{\mathbf{x} \sim \Psi^2} [E_L(\mathbf{x})] \right) \nabla_{\theta} \log \Psi(\mathbf{x}) \right]$$

1.4.2=4.2 NES-VMC 梯度

对于总 Ansatz，梯度计算类似。定义对数幅度：

$$\log |\Psi(\mathbf{x})| = \log \det(\Psi(\mathbf{x})) = \mathrm{Tr} \left(\log(\Psi(\mathbf{x})) \right)$$

梯度公式为：

$$\nabla_{\theta} \mathcal{L} = 2 \mathbb{E}_{\mathbf{x} \sim \Psi^2} \left[\left(E_L(\mathbf{x}) - \bar{E}_L \right) \nabla_{\theta} \log |\Psi(\mathbf{x})| \right]$$

其中 $\bar{E}_L = \mathbb{E}_{\mathbf{x} \sim \Psi^2} [E_L(\mathbf{x})]$ 是局域能量矩阵的期望值。

1.4.3=4.3 批量 WALKER 的梯度估计

与标准 VMC 类似，可以使用同一批次中独立的 walker 来获得无偏梯度估计：

$$\nabla_{\theta} \mathcal{L} = \frac{N-1}{2N} \mathbb{E}_{\mathbf{x}_1, \dots, \mathbf{x}_N} \left[\frac{1}{N} \sum_{i=1}^N \left(E_L(\mathbf{x}_i) - \frac{1}{N} \sum_{j=1}^N E_L(\mathbf{x}_j) \right) \nabla_{\theta} \log \Psi(\mathbf{x}_i) \right]$$

1.5=5. 激发态能量提取

1.5.1=5.1 能量矩阵的对角化

训练完成后，通过大量采样累积局域能量矩阵：

$$\bar{E}_L = \mathbb{E}[\langle \Psi^2 | E_L | \Psi^2 \rangle]$$

然后对 $\bar{E}_L$ 进行对角化：

$$\bar{E}_L = U \Lambda U^{-1}$$

其中 $\Lambda = \text{diag}(E_1, E_2, \dots, E_K)$ 包含按能量排序的本征值。

1.5.2=5.2 物理解释

当单态 Ansatz 是本征函数的线性组合 $\psi_i = \sum_j a_{ij} \psi_j^{\text{star}}$ 时，有：

$$\hat{H} \Psi = A^{-1} \Lambda A \Psi$$

其中 A 是系数矩阵。因此，通过对角化可以直接获得各激发态的能量 $E_1, E_2, \dots, E_K$ 。

1.6=6. 技术实现演进与问题排查

以下代码不要修改： 其中的希尔伯尔空间已经被拓展了 hi_extended 对其进行采样的话 我认为就是原文中说的"一次性采样K组组态"

```
import jax
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import optax
import numpy as np
from pyscf import gto, scf, fci
import flax.linen as nn

# 禁用 JIT 调试 (正式运行时可注释掉)
# jax.config.update('jax_disable_jit', True)

# =====
# 1. H2 分子与 FCI 基准
# =====
K = 2
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
mf = scf.RHF(mol).run(verbose=0)
```

```

cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, _ = cisolver.kernel()
print("=" * 60)
print("FCI 基准能量 (Ha)")
for i, e in enumerate(E_fcis):
    print(f"E{i} = {e:.8f} | 激发能 = {(e - E_fcis[0]) * 27.2114:.4f} eV")

ha = nkx.operator.from_pyscf_molecule(mol)

# =====
# 2. 扩展希尔伯特空间与采样器
# =====
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1, 1)
)
hi_extented = hi ** K

edges = [(0, 1), (2, 3)]
g = nk.graph.Graph(edges=edges)
single_rule = nk.sampler.rules.FermionHopRule(hi, graph=g)
tensor_rule = nk.sampler.rules.TensorRule(hi_ext, [single_rule] * K)
sampler = nk.sampler.MetropolisSampler(hi_ext, rule=single_rule, n_chains=16, sweep_size=32)

class SingleStateAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals: int, hidden_dim=16, *, rngs: nnx.Rngs):
        super().__init__()
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x):
        h = nnx.tanh(self.linear1(x.astype(complex)))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        return jnp.squeeze(out)

```

接下来是我对 Total WaveFunction 的定义:

```

class TotalAnsatz(nnx.Module):
    """NES 总 Ansatz: 输入 K 个副本状态, 返回  $\det[\psi_j(x^i)]$ """
    def __init__(self, n_spin_orbitals: int, n_states: int = K, hidden_dim: int = 16):
        super().__init__()
        self.n_states = n_states
        self.n_spin_orbitals = n_spin_orbitals

        key = jax.random.key(42)
        keys = jax.random.split(key, n_states)

        self.single_ansatz_list = [
            SingleStateAnsatz(
                n_spin_orbitals,
                hidden_dim,
                rngs=nnx.Rngs(keys[i])
            )
            for i in range(n_states)
        ]

    def __call__(self, x_batch):
        """
        x_batch: (n_states, n_spin_orbitals) -> (psi_total, M)

```

```

        M_{ij} = \psi_j(x^i)
        """
        K_state = self.n_states
        M = []
        for i in range(K_state):
            row = [self.single_ansatz_list[j](x_batch[i]) for j in range(K_state)]
            M.append(jnp.array(row))
        M = jnp.stack(M)
        psi_total = jnp.linalg.det(M)
        return psi_total, M

# 实例化 TotalAnsatz
total_ansatz = TotalAnsatz(
    n_spin_orbitals=n_spin_orbitals,
    n_states=K,
    hidden_dim=16,
)

graph_def, params = nnx.split(total_ansatz)

```

为了计算 $\mathcal{H}\Psi(X)$